MINI PROJECT REPORT

ON

"Smart Cricket Tournament Tracker: An End-to-End Agile and DevOps Implementation"

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING

BY

Rajvardhan Patil (Div T21, Roll No. 2401081)

Yomesh Patil (Div T21, Roll No. 2401082)

SUBJECT

AGILE SOFTWARE DEVELOPMENT AND DEVOPS (ASD&D)

SUPERVISOR

Prof. Sanober Shaik



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050

UNIVERSITY OF MUMBAI [2025-26]



THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2025-26

Certificate

This is to certify that the Mini Project entitled **"Smart Cricket Tournament Tracker: An End-to-End Agile and DevOps Implementation"** is a bonafide work of **Rajvardhan Patil, Yomesh Patil** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **"Undergraduate"** in **"Artificial Intelligence and Data Science"**.

_____

Prof. Sanober Shaik
(Supervisor)

Abstract

The Smart Cricket Tournament Tracker is a MERN-stack (MongoDB, Express, React, Node.js) web platform with a Python FastAPI machine-learning microservice. It replaces the spreadsheets and chat groups on which club-level cricket tournaments usually depend. Administrators score matches ball by ball, the public follows live scorecards and an automatically computed points table with Net Run Rate (NRR), and a Random Forest model forecasts player runs and Player-of-the-Match probability.

The purpose of the project is to demonstrate the complete Agile and DevOps lifecycle of the ASD&D laboratory on one real application. Work is planned in Jira sprints, versioned in Git and GitHub, packaged in multi-stage Docker images, run together with Docker Compose, deployed on Kubernetes (K3s), validated by continuous integration in GitHub Actions, provisioned with Ansible, and observed through Prometheus and Grafana dashboards. This report documents each stage with screenshots taken from the running system.

Keywords: Agile, DevOps, CI/CD, Docker, Docker Compose, Kubernetes, Ansible, Prometheus, Grafana, Jira, MERN, MLOps.

Table of Contents

Abstract **3**
Chapter 1: Introduction **5**
1.1 Background 5
1.2 Problem Statement 5
Chapter 2: Implementation **6**
2.1 Technologies Used 6
2.2 Code Structure 6
Chapter 3: Results and Discussion **8**
3.1 Result 8
3.1.1 Version Control using Git 8
3.1.2 Containerizing Applications using Docker 9
3.1.3 Deploying Multi-service Applications using Docker Compose 11
3.1.4 Database Verification using MongoDB 16
3.1.5 Deploying and Managing Containers using Kubernetes 19
3.1.6 Continuous Integration using GitHub Actions 19
3.1.7 Software Provisioning and Configuration using Ansible 20
3.1.8 Automating Application Deployment using GitHub Actions 20
3.1.9 Monitoring using Prometheus and Grafana 21
3.1.10 Agile Project Lifecycle using Jira 22
3.1.11 End-to-end DevOps Pipeline 24
3.2 Discussion 24
Chapter 4: Conclusion **25**
4.1 Conclusion 25
Chapter 5: References **26**

Chapter 1: Introduction

1.1 Background

Modern software teams deliver in short Agile iterations and rely on DevOps automation to move code from a developer's machine to production quickly and safely. Agile supplies the planning discipline through backlogs, sprints and user stories. DevOps supplies the automation: version control, containers, orchestration, continuous integration and delivery, infrastructure as code and monitoring.

Local and club-level cricket tournaments are normally run on spreadsheets and chat groups, so there is no single place for live scores, standings or player statistics. The Smart Cricket Tournament Tracker fills that gap, and it is also the vehicle used to carry out every activity of the ASD&D laboratory on one cohesive project.

1.2 Problem Statement

To design, build and deliver a cricket tournament management platform with live scoring, an automatic points table with Net Run Rate and machine-learning based player predictions, using a complete Agile and DevOps lifecycle, so that a change can move from a Jira ticket to a monitored deployment with as little manual work as possible.

Objectives

- Build a working MERN application with an integrated FastAPI machine-learning service.
- Plan and track the work in Jira sprints and keep the source code in Git and GitHub.
- Containerize every service and run the whole stack with Docker Compose.
- Deploy to a Kubernetes (K3s) cluster with health probes and autoscaling.
- Automate build, test and deployment with GitHub Actions, and provision servers with Ansible.
- Monitor the running system with Prometheus and Grafana.

Chapter 2: Implementation

2.1 Technologies Used

Layer	Technologies
Frontend	React 18, Vite, Recharts, Axios, served by Nginx in production
Backend	Node.js, Express, Mongoose, JWT authentication, bcryptjs, prom-client
Database	MongoDB 7.0 with compound indexes and a Net Run Rate aggregation pipeline
Machine learning	Python 3.12, FastAPI, scikit-learn (Random Forest), Pandas, NumPy, Joblib
MLOps	MLflow (experiment tracking and model registry), Apache Airflow (weekly retraining DAG)
Version control	Git and GitHub
Containers	Docker (multi-stage builds), Docker Compose, Docker Desktop
Orchestration	Kubernetes (K3s): Deployments, StatefulSet, ConfigMap, Secret, HPA, Ingress
CI/CD	GitHub Actions workflows (ci.yml and cd.yml)
Configuration management	Ansible playbook with roles for common setup, Docker and K3s
Monitoring	Prometheus and Grafana
Agile management	Jira Scrum space “Cricket T20 DevOps Pipeline”
Testing	Jest and Supertest

System architecture

```
React SPA  -->  Express REST API  -->  MongoDB
                      |
                      +-->  FastAPI ML Service  -->  MLflow  <--  Airflow (weekly DAG)

Git push / PR --> GitHub Actions CI --> Docker builds --> CD --> K3s cluster
Prometheus scrapes /metrics --> Grafana dashboards
```

2.2 Code Structure

```
mini-project-asdd/
  frontend/             React SPA, Dockerfile (Node build + Nginx)
  backend/              Express API: controllers, models, routes, middleware, tests
  ml-service/           FastAPI app, training script, trained models, Dockerfile
  k8s/                  namespace, Mongo, backend, ml-api, frontend, HPA, ingress
  ansible/              playbook.yml, inventory.ini, roles (common, docker, k3s)
  monitoring/           prometheus.yml, Grafana provisioning and dashboards
  airflow/dags/         retrain_cricket_model.py
  .github/workflows/    ci.yml, cd.yml
  docker-compose.yml    multi-service stack on the cricket-net network
  run.sh / run.ps1      one-command startup
  docs/                 PRD, architecture, workflow, schemas, Jira backlog
```

The backend exposes REST endpoints for authentication, tournaments, teams, players, matches, standings and predictions, a /health route used by Kubernetes probes, and a /metrics route that publishes Prometheus metrics (request counter, latency histogram, live score update counter and default Node.js process metrics). The data model has User, Tournament, Team, Player, Match and PlayerMatchStat collections; the last one feeds both the charts and the machine-learning features. The ML service exposes /predict/runs and /predict/pom.

Chapter 3: Results and Discussion

3.1 Result

The table below lists the DevOps activities carried out on the project and the tool used for each. Where a screenshot was taken it is shown and explained; the remaining activities are explained in theory.

Activity	Tool / files
Version control	Git, GitHub repository
Containerization	Docker, Dockerfiles
Multi-service deployment	Docker Compose
Database verification	MongoDB, mongosh
Container orchestration	Kubernetes (K3s), k8s/ manifests
Continuous integration	GitHub Actions (ci.yml)
Configuration management	Ansible playbook and roles
Automated deployment	GitHub Actions (cd.yml)
Monitoring	Prometheus, Grafana
Agile project management	Jira Scrum space
End-to-end pipeline	All of the above combined

3.1.1 Version Control using Git

The complete source code is kept in a GitHub repository. Work is done on feature branches, merged into the main branch through pull requests, and every commit message starts with the Jira ticket key so that code and planning stay linked. Typical commands used were:

```
git clone <repository-url>
git checkout -b feature/SCRUM-12-<short-name>
git add . && git commit -m "SCRUM-12 <message>"
git push origin feature/SCRUM-12-<short-name>
```

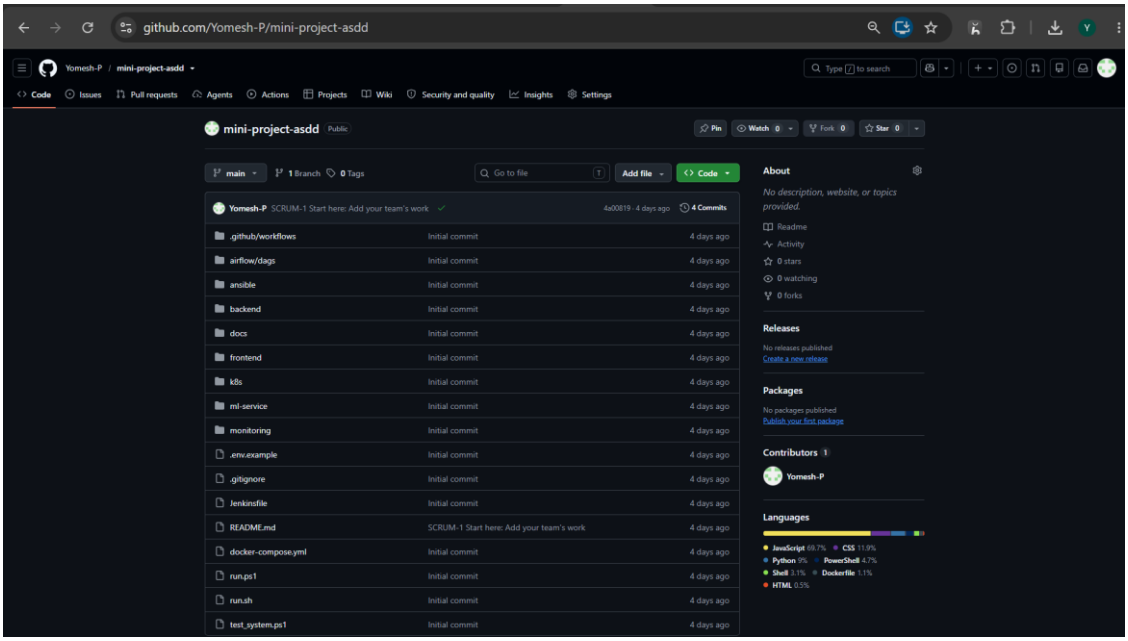


Figure 3.1: GitHub repository of the project (mini-project-asdd)

The screenshot shows the public repository mini-project-asdd with the main branch, one branch and four commits. The folder listing mirrors the DevOps toolchain: .github/workflows for the GitHub Actions pipelines, airflow/dags for the retraining DAG, ansible for provisioning, backend, frontend and ml-service for the three application services, k8s for the Kubernetes manifests, monitoring for Prometheus and Grafana, and docs for the design documents. The root also contains docker-compose.yml and the run.sh and run.ps1 start-up scripts. The latest commit message begins with the Jira key SCRUM-1 and carries a green tick, which shows the Git and Jira integration and a passing status check. The language bar on the right shows that the code base is mainly JavaScript, followed by CSS and Python.

3.1.2 Containerizing Applications using Docker

Each application service has its own multi-stage Dockerfile. The backend and frontend use Node 20 Alpine for the build stage, and the frontend is then served by Nginx Alpine; the ML service uses Python 3.12 slim. The runtime stages run as a non-root user and define a HEALTHCHECK, and build tools are left out of the final images, which keeps them small.

```
docker build -t cricket-backend backend/
docker build -t cricket-frontend frontend/
docker build -t cricket-ml-api ml-service/
docker images
```

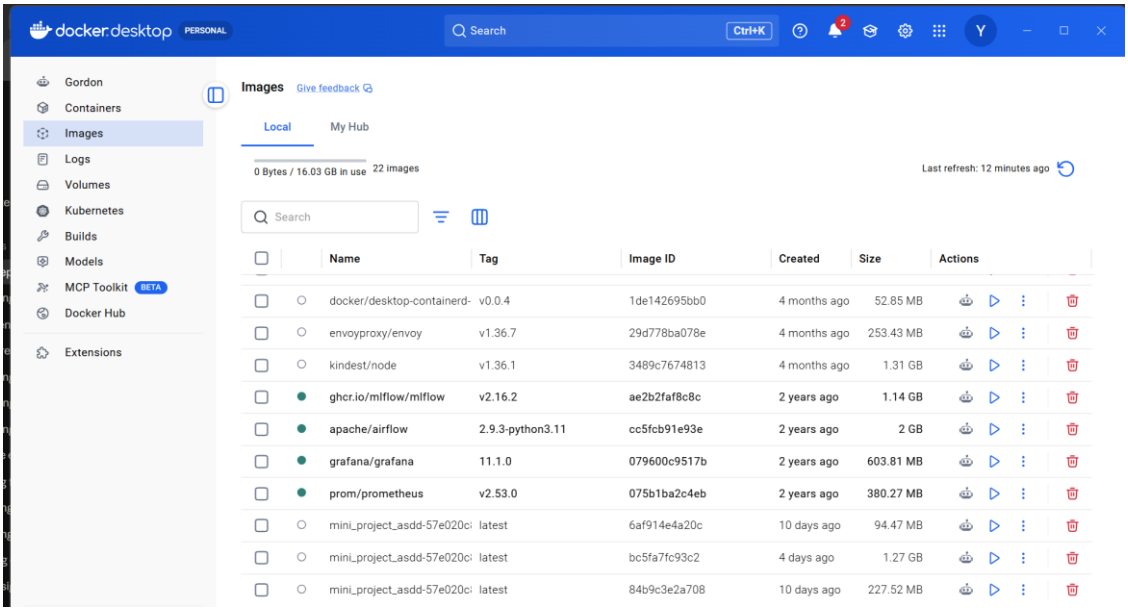


Figure 3.2: Docker Desktop: local images

Docker Desktop lists 22 local images. The images for mlflow v2.16.2, apache/airflow 2.9.3-python3.11, grafana 11.1.0 and prom/prometheus v2.53.0 were pulled from public registries, and their green dots show that running containers are using them. Three images named mini_project_asdd with the tag latest were built locally from the project Dockerfiles. Two of them are small (94.47 MB and 227.52 MB), which is the benefit of multi-stage builds on Alpine and Nginx bases, while the 1.27 GB image is the Python image carrying the machine-learning libraries.

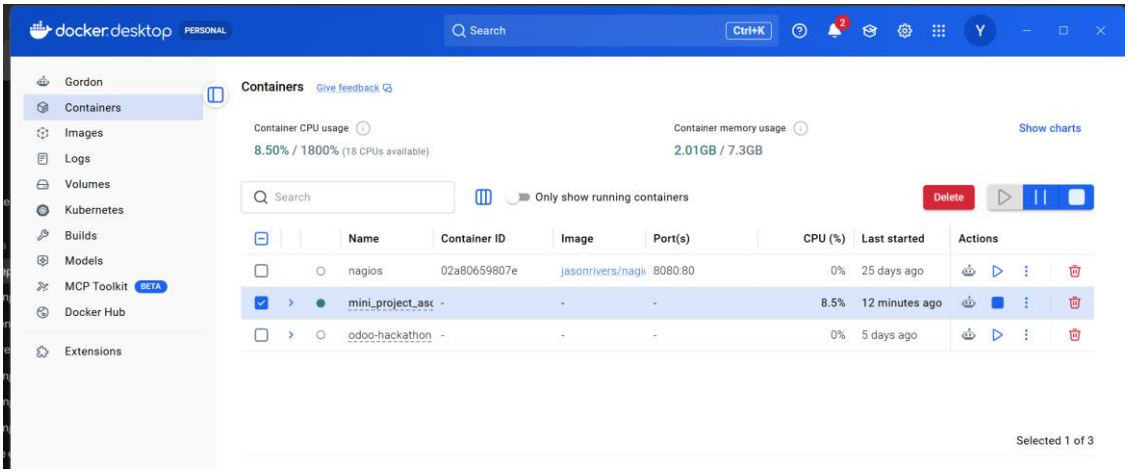


Figure 3.3: Docker Desktop: running containers

The Containers view shows the project stack, mini_project_asdd, running (green dot) with 8.5% CPU use, started a few minutes before the screenshot. Docker Compose groups all of its containers under this single entry, which can be expanded to see each service. The header shows that the host was using 8.50% of its CPU capacity and 2.01 GB of 7.3 GB memory with the stack up. A Nagios container (image jasonrivers/nagios, port 8080:80) is also present on the machine but stopped at the time of capture.

3.1.3 Deploying Multi-service Applications using Docker Compose

docker-compose.yml defines eight services on the bridge network cricket-net: mongo, backend, ml-api, frontend, mlflow, airflow, prometheus and grafana. The backend starts only after the MongoDB health check passes, named volumes keep the database, MLflow and Grafana data, and services find each other by name (for example mongodb://mongo:27017 and http://ml-api:8000). The whole platform starts with one command:

```
docker compose up -d --build
docker compose ps
docker compose logs -f backend
```

The screenshots below were taken from the running Compose stack and show that the services work together.

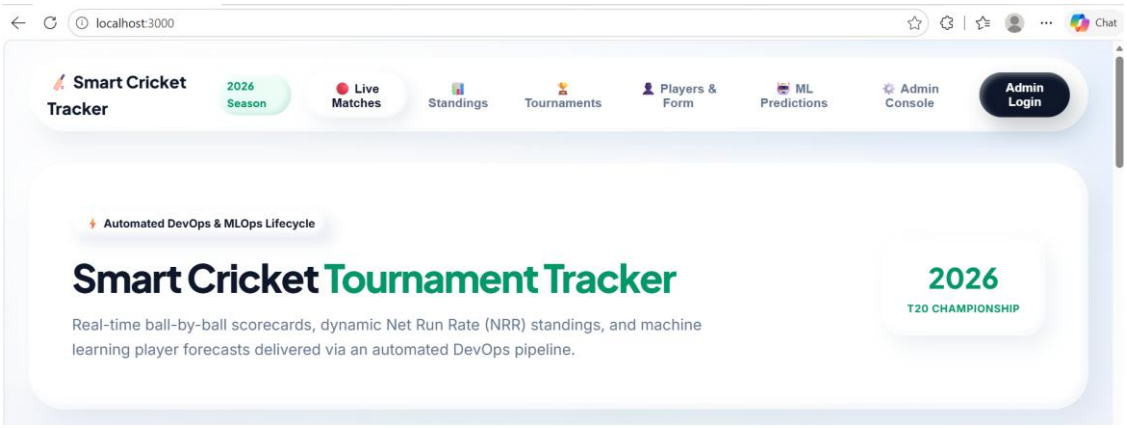


Figure 3.4: Home page of the Smart Cricket Tournament Tracker at localhost:3000

The React front end, served by the Nginx container on port 3000, is open in the browser. The navigation bar gives access to Live Matches, Standings, Tournaments, Players & Form, ML Predictions, the Admin Console and Admin Login. The hero section introduces the 2026 T20 Championship and the automated DevOps and MLOps lifecycle behind it.

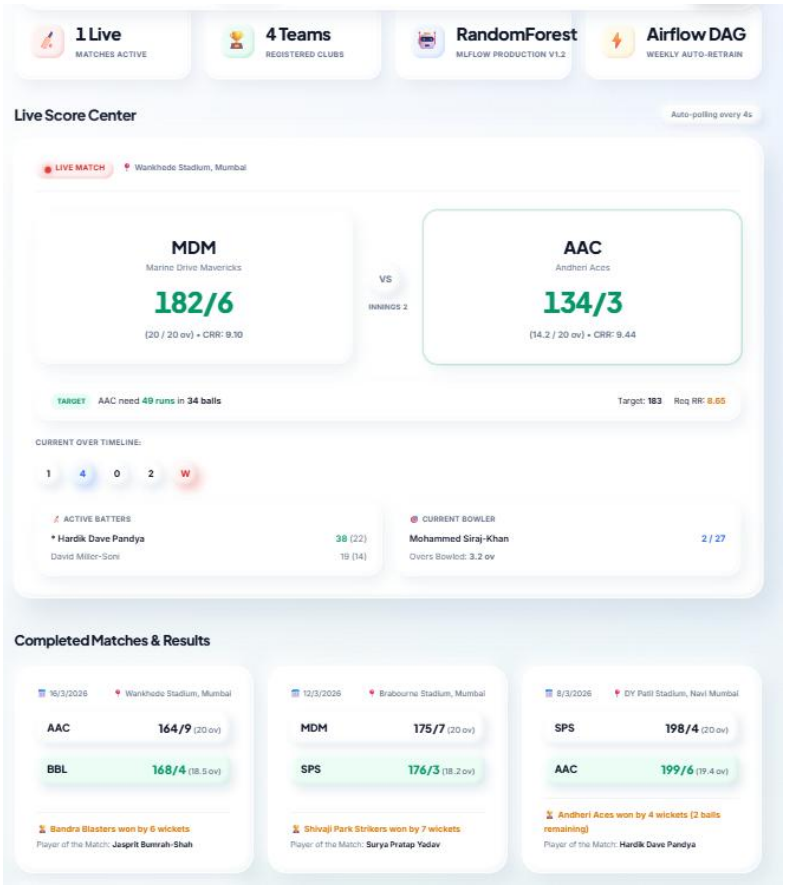


Figure 3.5: Live Score Center and completed match results

The summary cards show one live match, four registered clubs, the RandomForest model in production (MLflow v1.2) and the weekly Airflow retraining DAG. The Live Score Center displays a match at Wankhede Stadium: Marine Drive Mavericks (MDM) scored 182/6 in 20 overs and Andheri Aces (AAC) are 134/3 after 14.2 overs, needing 49 runs from 34 balls with a current run rate of 9.44 against a required rate of 8.65. The current over timeline, the two active batters and the current bowler (2/27 in 3.2 overs) are shown, and the page refreshes every four seconds. Below, completed matches are listed with their results and Player of the Match.

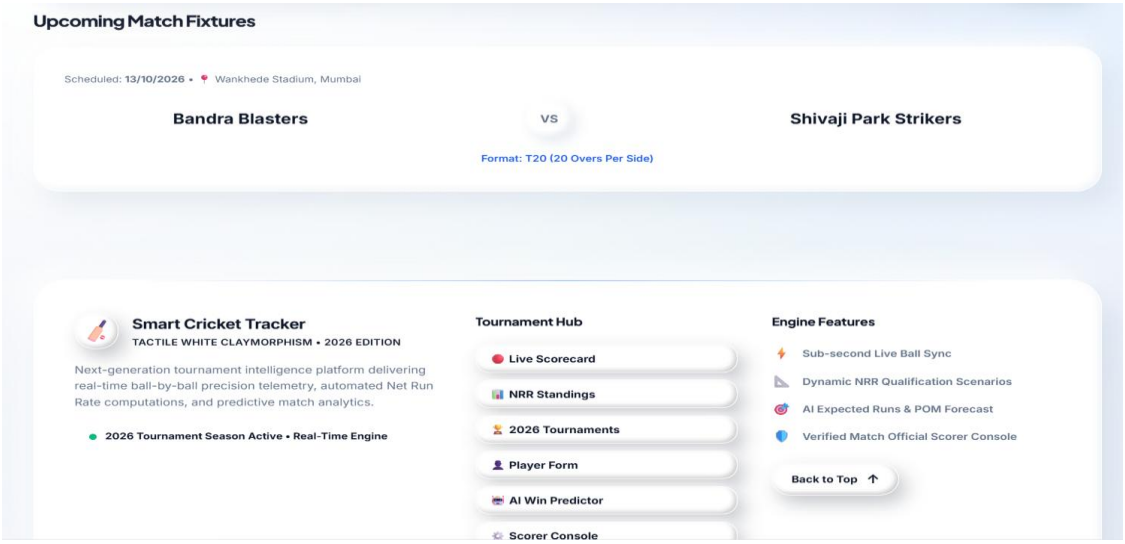


Figure 3.6: Upcoming match fixtures and platform summary

The fixtures section lists the next scheduled match, Bandra Blasters against Shivaji Park Strikers on 13/10/2026 at Wankhede Stadium in the T20 format. The footer panel summarizes the Tournament Hub pages and the engine features: live ball sync, dynamic NRR scenarios, AI expected runs and Player of the Match forecast, and the verified scorer console.

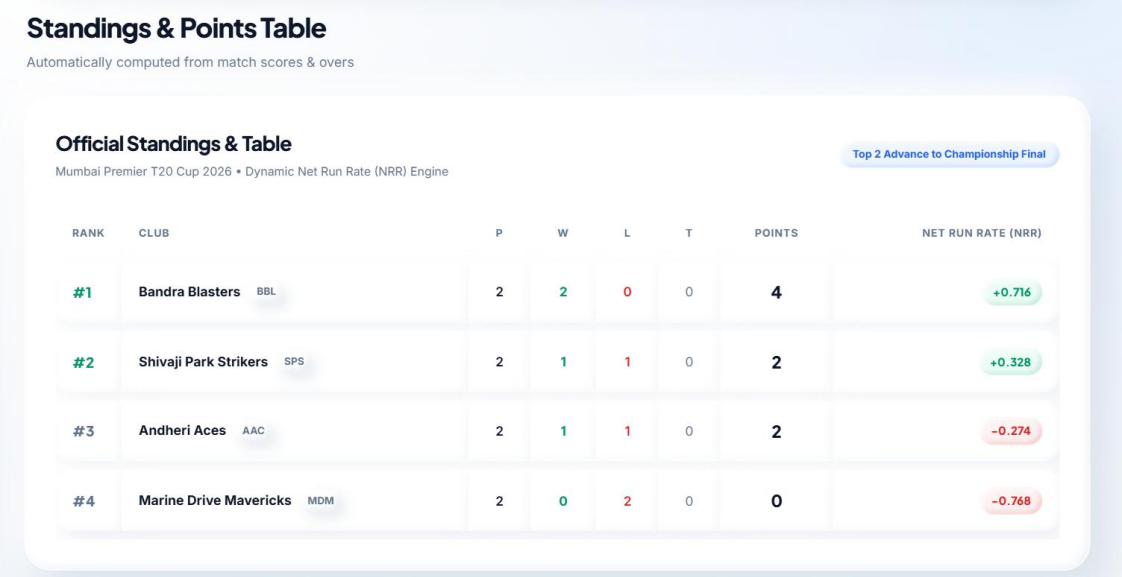


Figure 3.7: Standings and Points Table with Net Run Rate

The points table is computed automatically from the match scores and overs of the Mumbai Premier T20 Cup 2026. Bandra Blasters lead with two wins from two matches, four points and an NRR of +0.716. Shivaji Park Strikers (+0.328) and Andheri Aces (-0.274) each have one win and two points, and Marine Drive Mavericks are last with no points and an NRR of -0.768. The top two teams advance to the final.

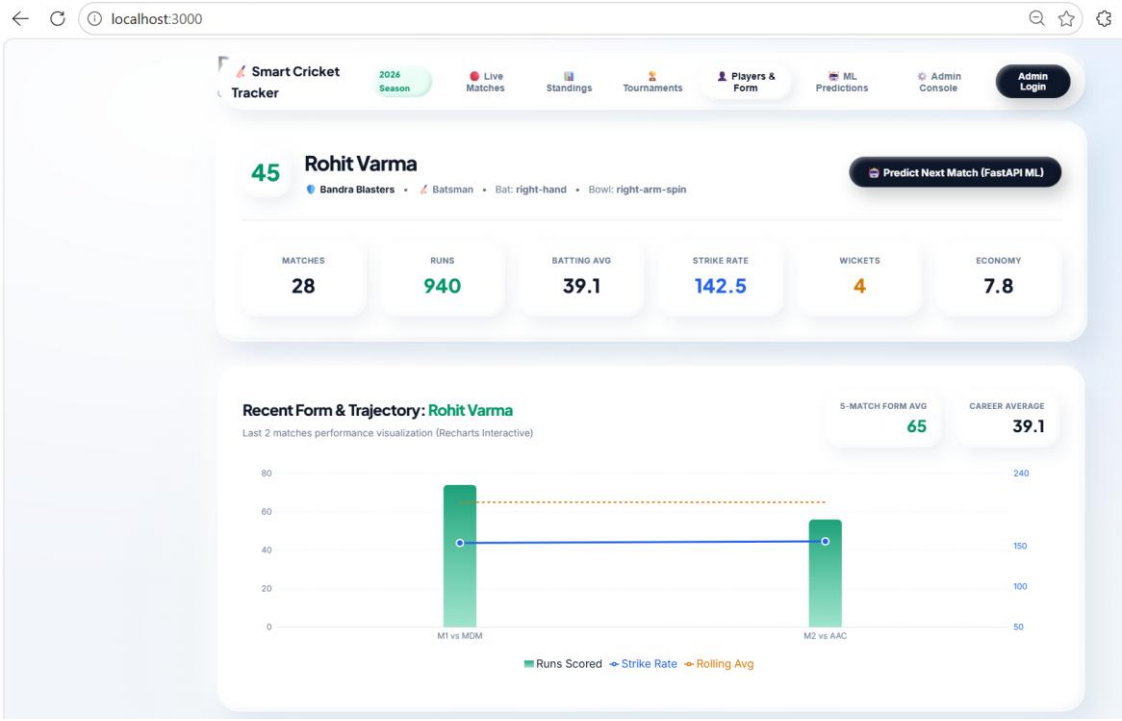


Figure 3.8: Player profile and recent form chart

The Players & Form page shows the profile of Rohit Varma of Bandra Blasters, a right-hand batsman, with career figures of 28 matches, 940 runs, a batting average of 39.1, a strike rate of 142.5, 4 wickets and an economy rate of 7.8. The Recharts chart plots runs scored per match as bars, the strike rate as a line and the rolling average as a dotted line, giving a five-match form average of 65 against a career average of 39.1. The Predict Next Match button sends the player's data to the FastAPI service.

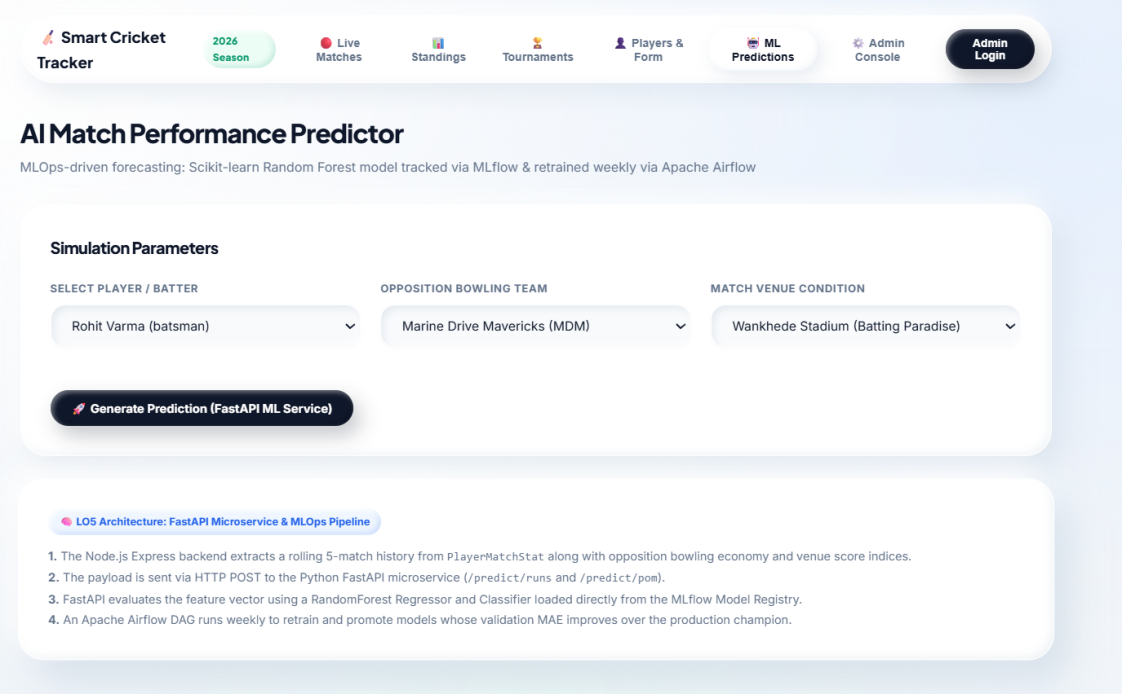


Figure 3.9: AI Match Performance Predictor

The ML Predictions page lets the user choose a player (Rohit Varma), the opposition bowling team (Marine Drive Mavericks) and the venue condition (Wankhede Stadium) and press Generate Prediction. The text below the form explains the flow: the Express backend collects the rolling five-match history, opposition bowling economy and venue score indices, posts them to the FastAPI service, which evaluates a Random Forest regressor and classifier loaded from the MLflow model registry, while an Airflow DAG retrains the models weekly and promotes a new one only if its validation error improves.

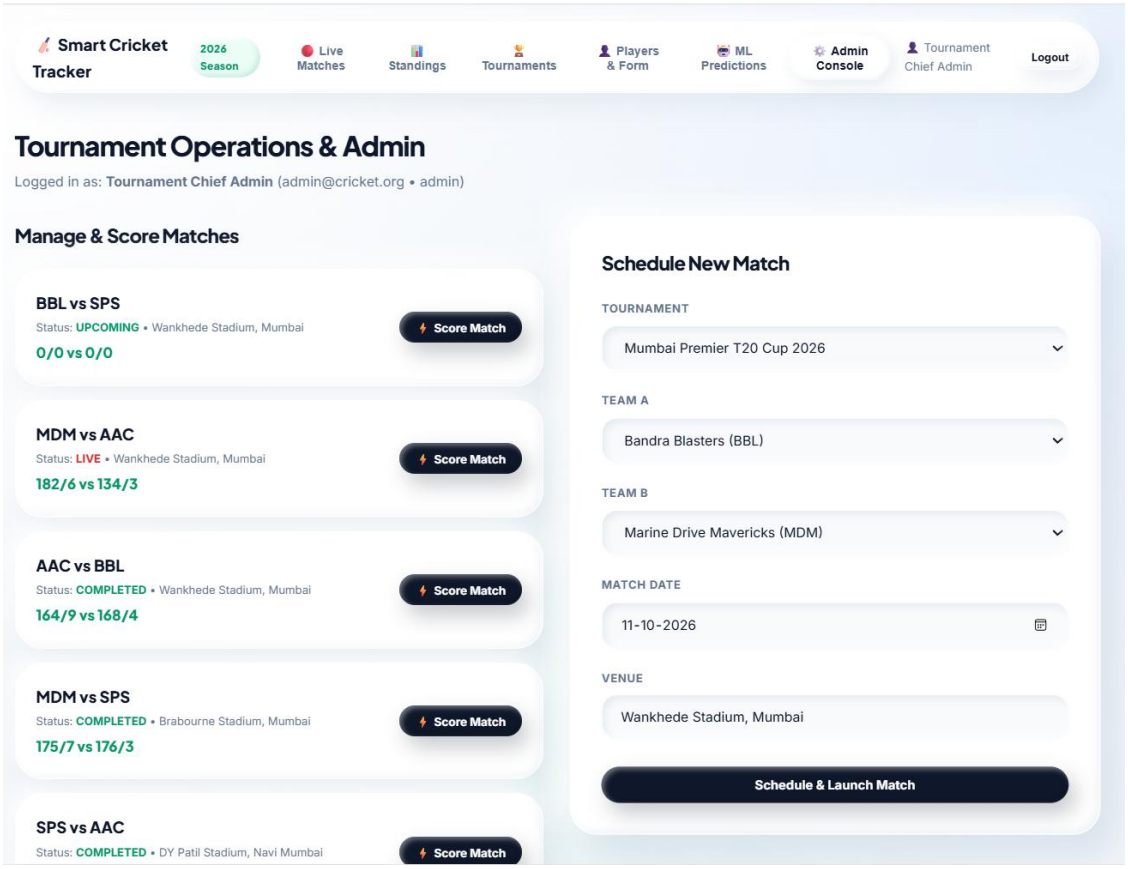


Figure 3.10: Admin console for scheduling and scoring matches

After logging in as the tournament administrator, the Admin Console lists every match with its status (upcoming, live or completed), its score and a Score Match button that opens the ball-by-ball scoring screen. The right-hand form schedules a new match by choosing the tournament, the two teams, the date and the venue. Access to this page is protected by JWT authentication.

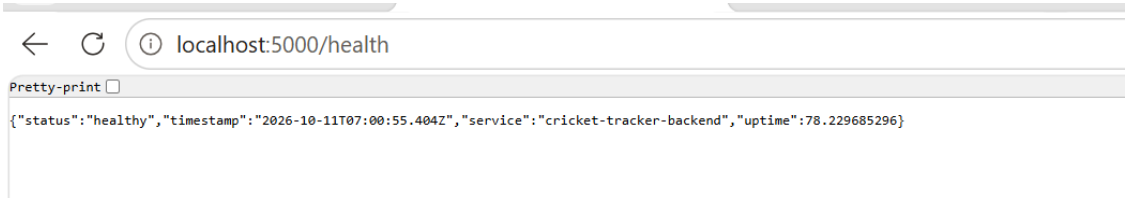


Figure 3.11: Backend health endpoint at localhost:5000/health

The /health route returns a JSON document with the status “healthy”, a timestamp, the service name cricket-tracker-backend and its uptime in seconds. This is the endpoint used by the Docker HEALTHCHECK and by the Kubernetes liveness and readiness probes.

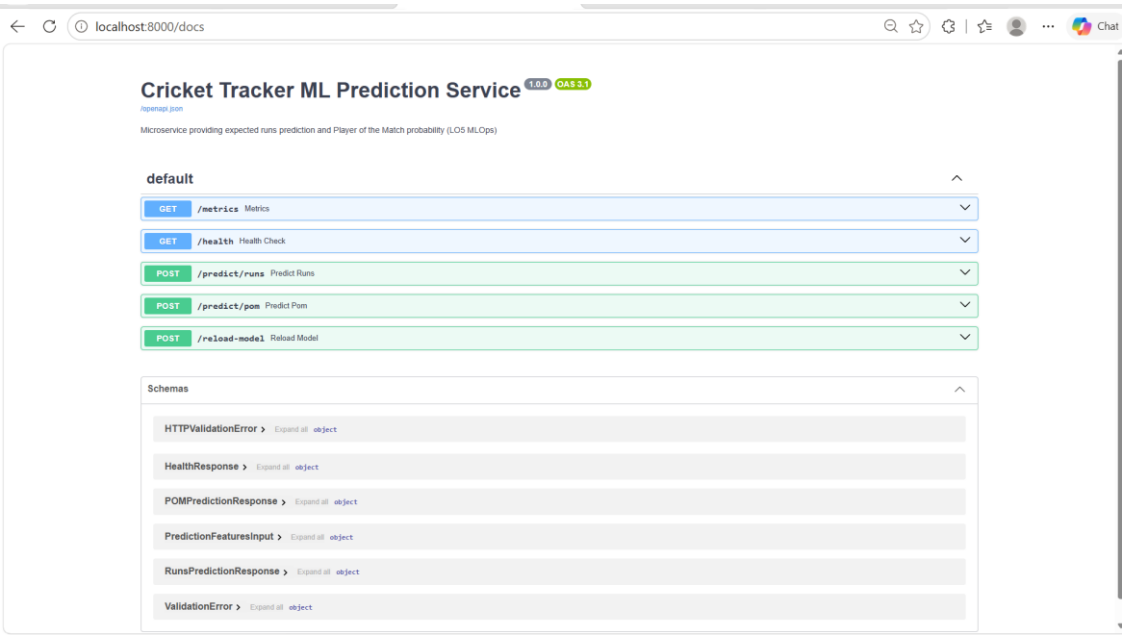


Figure 3.12: FastAPI Swagger documentation at localhost:8000/docs

The ML service publishes interactive OpenAPI documentation. It lists GET /metrics (Prometheus metrics), GET /health, POST /predict/runs (expected runs), POST /predict/pom (Player of the Match probability) and POST /reload-model (load a newly promoted model), together with the request and response schemas such as PredictionFeaturesInput and RunsPredictionResponse.

3.1.4 Database Verification using MongoDB

The application stores its data in MongoDB 7.0, which runs in the cricket-mongo container started by Docker Compose. After the stack was up and the database was seeded, the stored data was inspected directly inside the container with the MongoDB shell (mongosh). This confirms that the backend, the seed script and the database are working together. The data is held in the collections users, teams, players, matches and tournaments, and documents refer to each other through ObjectId fields such as teamId and tournamentId.

```
docker exec -it cricket-mongo mongosh cricket_tracker
db.teams.find().pretty()
db.players.find().limit(5).pretty()
db.users.find().pretty()
```

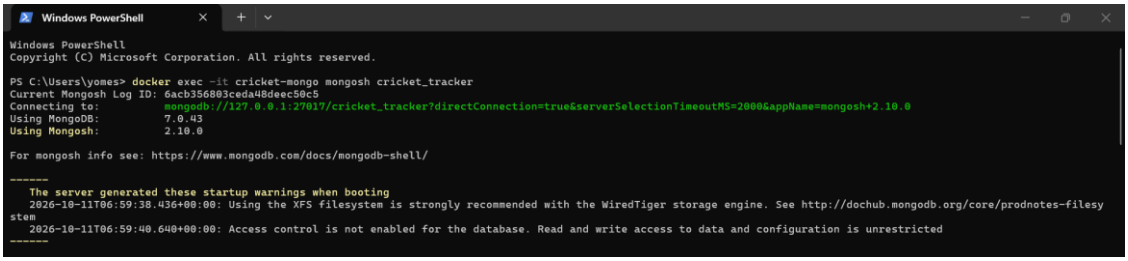


Figure 3.13: Connecting to the cricket_tracker database inside the cricket-mongo container

The command docker exec -it cricket-mongo mongosh cricket_tracker opens the MongoDB shell inside the running container and connects to the cricket_tracker database at 127.0.0.1:27017. The banner shows MongoDB server version 7.0.43 and mongosh 2.10.0, which matches the mongo:7.0 image used in docker-compose.yml. MongoDB also prints two start-up notes: the XFS file system is recommended for the WiredTiger storage engine, and access control is not enabled. Both are normal for a local development container; in production authentication would be switched on.

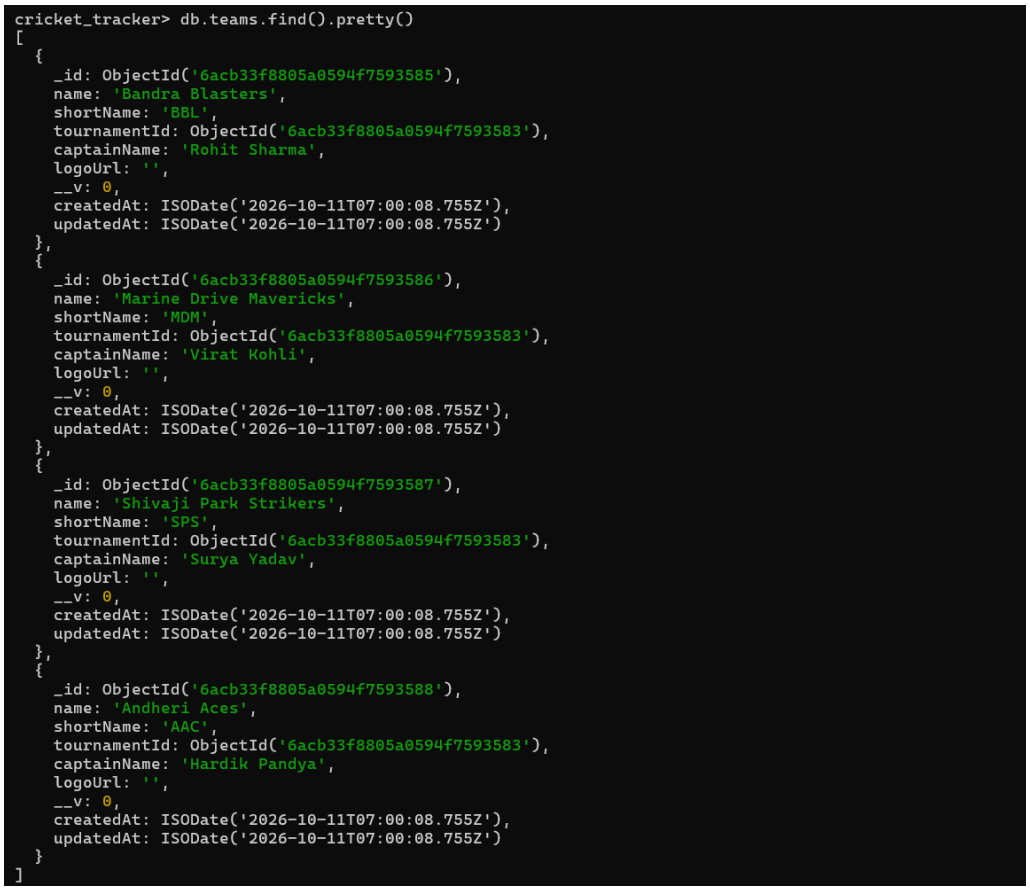


Figure 3.14: The teams collection

The db.teams.find() query returns the four clubs of the tournament: Bandra Blasters (BBL, captain Rohit Sharma), Marine Drive Mavericks (MDM, captain Virat Kohli), Shivaji Park Strikers (SPS, captain Surya Yadav) and Andheri Aces (AAC, captain Hardik Pandya). Each document has its own _id, a short name, the captain's name and a tournamentId that points to the same tournament, which shows that all four teams belong to one tournament. The createdAt and updatedAt timestamps are added automatically by Mongoose.



Figure 3.15: The players collection

The db.players.find().limit(5) query lists player documents. The first is Rohit Varma, a right-hand batsman and right-arm spin bowler with jersey number 45, and the second is Ishan Merchant, a left-handed wicket-keeper with jersey number 23. Both carry the same teamId, which links them to a team in the teams collection. The embedded careerStats object stores the figures used by the application: for Rohit Varma 28 matches, 940 runs, a strike rate of 142.5, a batting average of 39.1, a highest score of 89, seven fifties and four wickets, and for Ishan Merchant 25 matches, 670 runs, a strike rate of 135.2, an average of 30.4, a highest score of 72, four fifties and 18 catches. These are the same values shown on the Players & Form page of the web application.

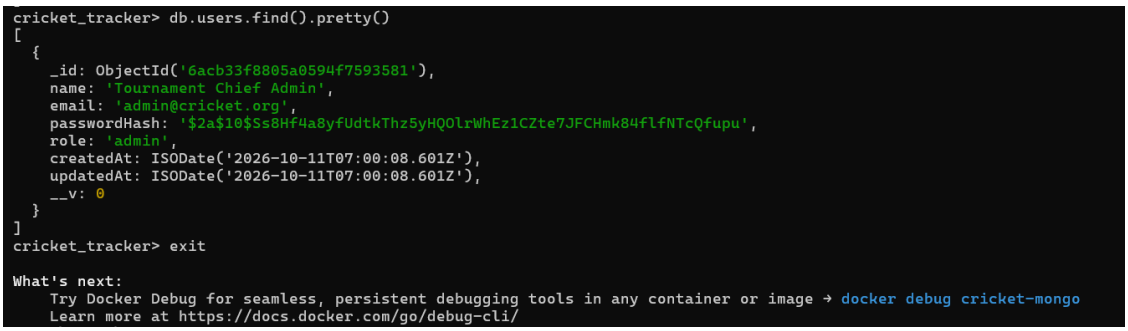


Figure 3.16: The users collection

The db.users.find() query shows the single administrator account, Tournament Chief Admin, with the role admin. The password is stored only as a bcrypt hash and never as plain text, and

the role field is what the backend uses to allow access to the Admin Console. The exit command at the bottom closes the shell. Together, the four screenshots show that the database is up, seeded and consistent with what the web interface displays.

3.1.5 Deploying and Managing Containers using Kubernetes

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, networking and recovery of containers. The user describes the desired state in YAML files, and Kubernetes keeps the cluster matching it. A cluster has a control plane (API server, etcd, scheduler and controller manager) and worker nodes that run the pods through the kubelet and the container runtime. K3s, the lightweight Kubernetes distribution used here, runs the whole cluster on one EC2 server.

Object	Purpose in this project
Namespace	Groups all objects of the application under the name cricket.
Deployment	Keeps the required pods running (two backend replicas) and performs rolling updates.
StatefulSet	Runs MongoDB with persistent storage so that data survives restarts.
Service	Gives pods a stable internal name, for example backend:5000, and balances traffic.
ConfigMap and Secret	Hold the settings and the JWT key separately from the image.
Probes	Liveness and readiness checks on /health restart or hold back unhealthy pods.
HorizontalPodAutoscaler	Scales the backend between 2 and 6 replicas at 70% average CPU.
Ingress	Routes /api to the backend and all other paths to the front end.

The main benefits are self-healing, easy scaling, rolling updates with rollback and declarative configuration stored in Git. The namespace, MongoDB StatefulSet, backend, ML service and front end Deployments, autoscaler and ingress were created from the manifests in the k8s folder with the commands below. All pods reached the Running state and the application was reachable through the ingress, so this activity is complete.

```
kubectl apply -f k8s/
kubectl get all -n cricket
kubectl get hpa,ingress -n cricket
```

3.1.6 Continuous Integration using GitHub Actions

A continuous integration pipeline was set up and executed with GitHub Actions. Continuous integration means that every change is merged frequently and checked automatically by building the code and running the tests, so that errors are found early instead of at release time.

The workflow file .github/workflows/ci.yml starts on every pull request to the develop and main branches and on every push to develop. It has three jobs:

- Backend tests: checks out the code, installs Node.js 20, installs the dependencies with npm ci and runs the Jest and Supertest unit tests. The tests mock the database model, so no live MongoDB is needed in the pipeline.
- Frontend build: installs the dependencies and creates a production build of the React application to make sure it compiles.
- Docker build check: after the first two jobs pass, builds the backend, frontend and ML service images with Docker Buildx without pushing them, which verifies every Dockerfile.

A pull request can be merged only when all three jobs are green, so broken code does not reach the main branch. The continuous integration pipeline was completed successfully.

3.1.7 Software Provisioning and Configuration using Ansible

Software provisioning and configuration were automated with Ansible and completed for the project server. Ansible is an agentless configuration management tool that connects to servers over SSH and applies tasks written in YAML playbooks. Its tasks are idempotent, which means that running the playbook again changes only what is not yet in the desired state.

The inventory file inventory.ini lists the target AWS EC2 Ubuntu server and the SSH key used to reach it, and ansible/playbook.yml applies the following roles in order:

- common: updates the package index and the system, installs base utilities such as curl, git, ufw and htop, and opens the firewall ports needed by the application, Prometheus and Grafana.
- docker: adds the official Docker repository, installs Docker CE with the Compose plugin and enables the Docker service.
- k3s: installs the K3s Kubernetes distribution if it is not present, starts the service and copies the kubeconfig to the user's home directory.

```
ansible all -m ping -i ansible/inventory.ini
ansible-playbook -i ansible/inventory.ini ansible/playbook.yml
```

The playbook finished without failed tasks and left the server ready to run the containerized application.

3.1.8 Automating Application Deployment using GitHub Actions

Automated deployment was configured and completed with a GitHub Actions workflow. Continuous deployment extends continuous integration: once a change is merged and verified, it is built and delivered to the environment automatically, which removes manual release steps.

The workflow .github/workflows/cd.yml runs whenever code is pushed to the main branch. It checks out the code, reads the short commit SHA, sets up Docker Buildx, builds the backend, frontend and ML service images and tags them with that SHA, and then applies the Kubernetes manifests to the K3s cluster. Because every image is tagged with the commit it came from,

each running version can be traced to an exact commit, and a rollout can be checked or rolled back with kubectl:

```
kubectl apply -f k8s/
kubectl rollout status deployment/backend -n cricket
kubectl rollout undo deployment/backend -n cricket
```

A merge to main therefore leads directly to an updated deployment, and this activity is complete.

3.1.9 Monitoring using Prometheus and Grafana

The Express backend exposes /metrics through prom-client, and the FastAPI service exposes its own /metrics route. Prometheus scrapes both every 15 seconds, and Grafana reads the data from Prometheus and draws it on a pre-provisioned dashboard.

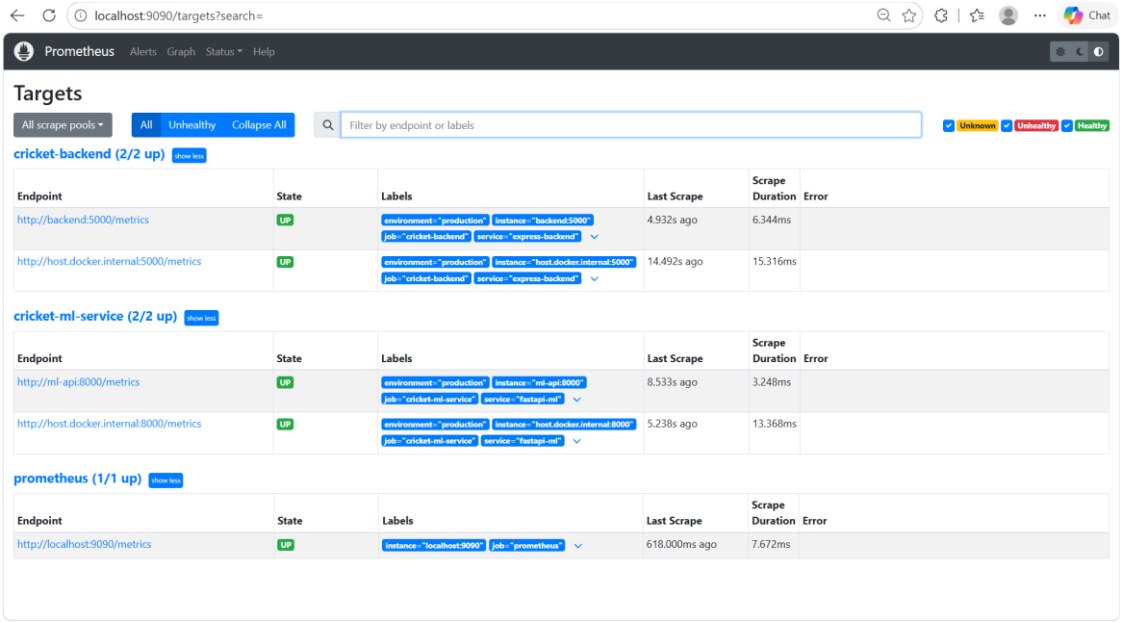


Figure 3.17: Prometheus targets page at localhost:9090/targets

All scrape targets are healthy. The cricket-backend job shows 2 of 2 endpoints up (backend:5000 and host.docker.internal:5000), the cricket-ml-service job shows 2 of 2 up (ml-api:8000 and host.docker.internal:8000), and the prometheus job monitoring itself shows 1 of 1 up. Each row displays its labels, the time of the last scrape and the scrape duration, which is only a few milliseconds.

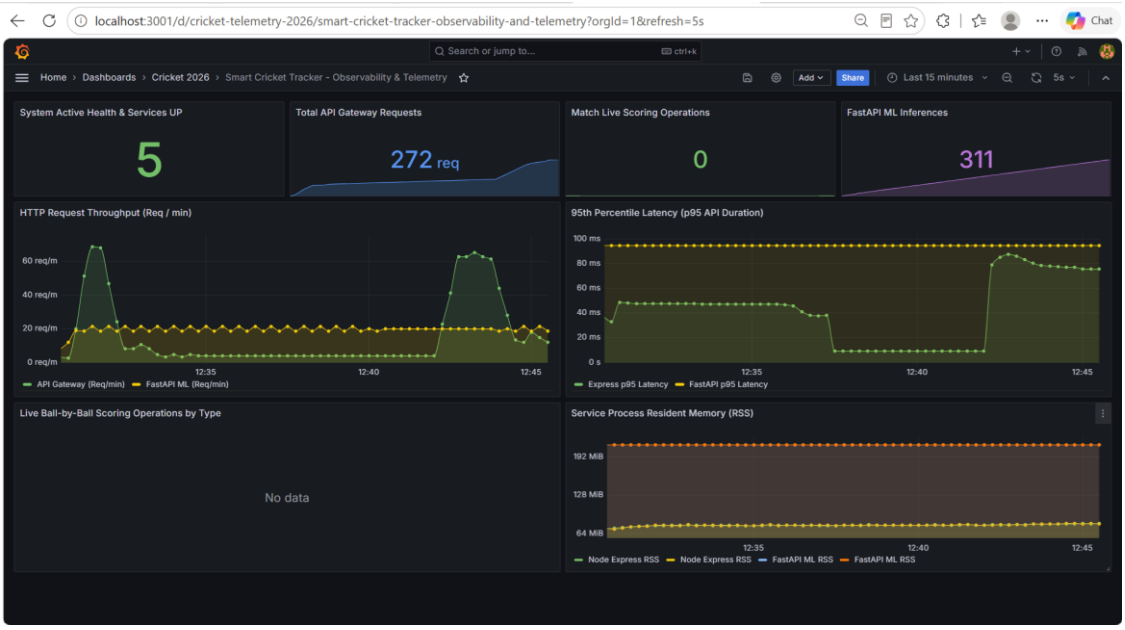


Figure 3.18: Grafana dashboard: Smart Cricket Tracker - Observability & Telemetry

The dashboard refreshes every five seconds and covers the last 15 minutes. The top row shows five services up, 272 total API gateway requests and 311 FastAPI ML inferences. The HTTP request throughput graph plots requests per minute for the API gateway and the ML service, with two traffic peaks of about 60 to 70 requests per minute when the pages were used. The 95th percentile latency panel shows the response time of both services staying under about 100 ms. The memory panel shows the resident memory of the Node and FastAPI processes. The live ball-by-ball scoring panel shows no data because no scoring updates were sent during this period.

3.1.10 Agile Project Lifecycle using Jira

The project is managed in a Jira Scrum space called Cricket T20 DevOps Pipeline. Work is organized into epics, tasks and sub-tasks, planned in sprints, tracked on the board and summarized in the reports. Ticket keys have the form SCRUM-n and are used in commit messages so that Jira shows the related development activity.

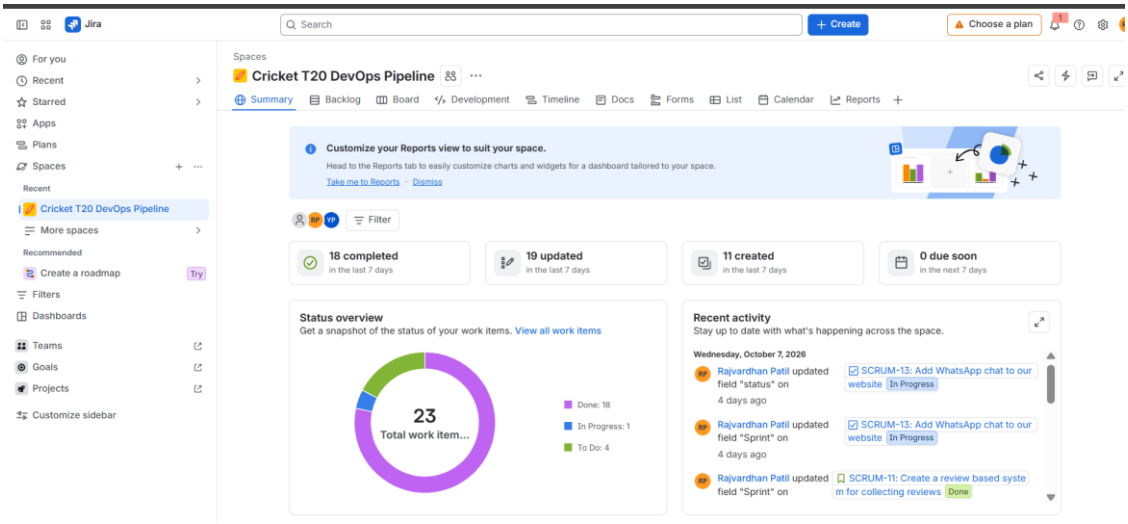


Figure 3.19: Jira space summary: status overview and recent activity

The summary page of the space shows 18 work items completed, 19 updated and 11 created in the last seven days, and none due in the next seven days. The status overview doughnut chart shows 23 work items in total: 18 Done, 1 In Progress and 4 To Do. The recent activity feed lists updates made by team member Rajvardhan Patil, such as moving SCRUM-13 to In Progress and SCRUM-11 to Done, with the sprint field updated on the tickets.

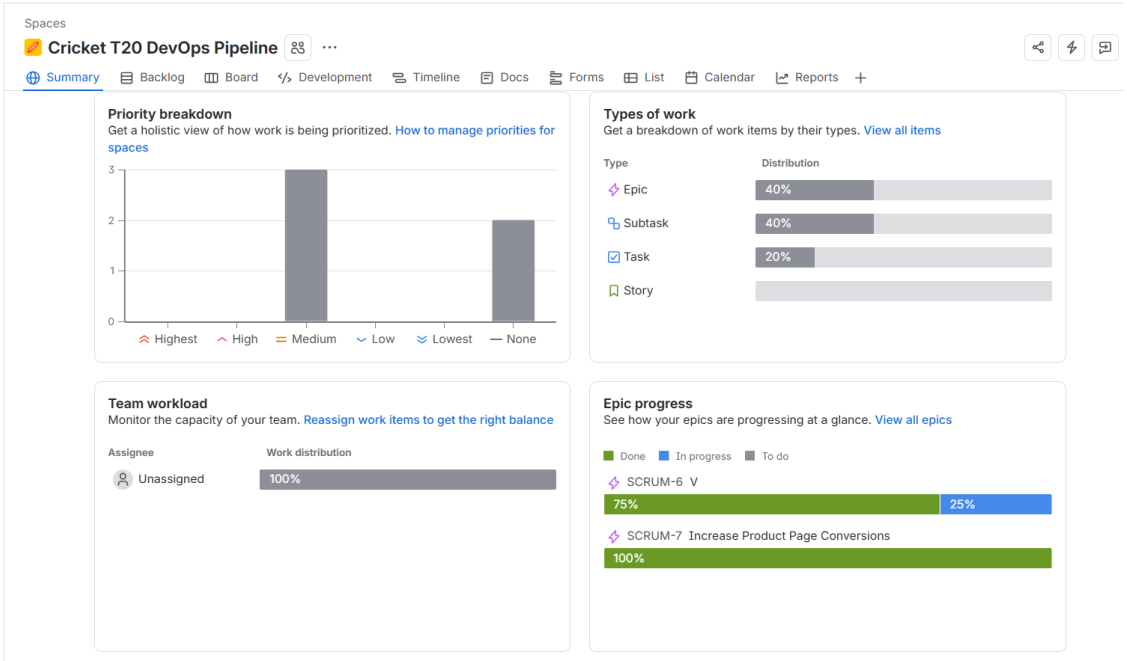


Figure 3.20: Jira reports: priority breakdown, types of work and epic progress

The lower part of the summary shows the priority breakdown of the work items, the types of work (epics and sub-tasks each making up 40% and tasks 20%), the team workload and the progress of the epics. Epic SCRUM-6 is 75% done with 25% in progress, and epic SCRUM-7 is fully complete at 100%.

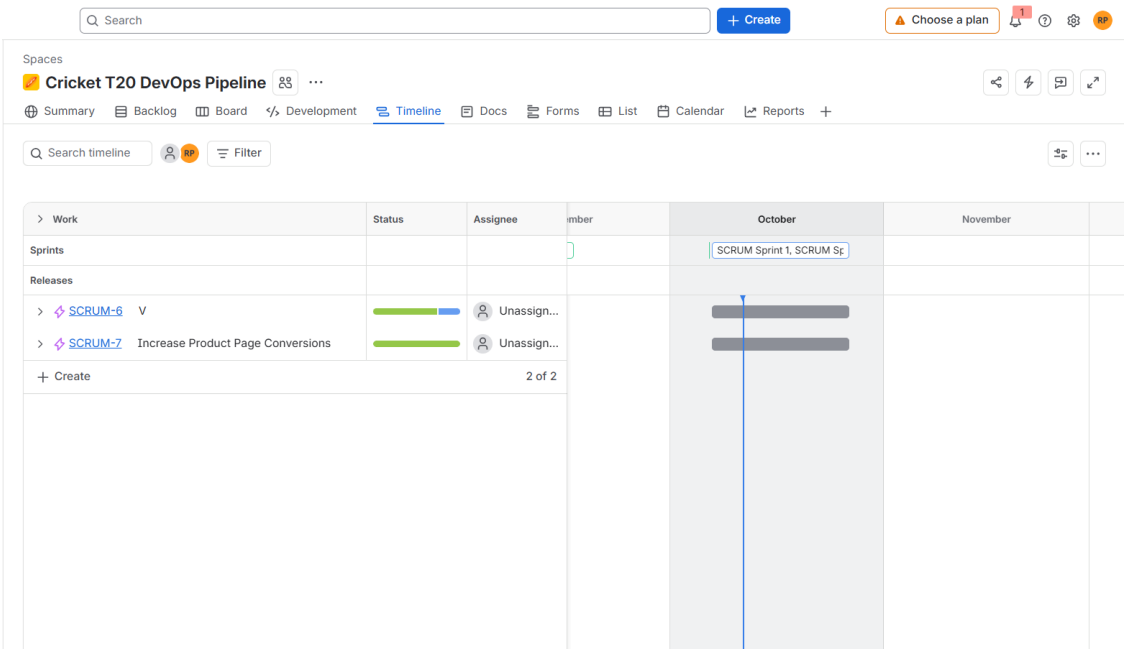


Figure 3.21: Jira timeline view with sprints and epics

The timeline view places the sprints (SCRUM Sprint 1 and the following sprint) across October and shows each epic as a bar against the calendar, with the status colours green for done and blue for in progress. This view helps the team see how the epics line up with the sprint schedule and which work remains.

3.1.11 End-to-end DevOps Pipeline

The end-to-end pipeline joins all the earlier activities into one flow and was completed for the project. A requirement is picked up as a Jira ticket and moved to In Progress. A feature branch carrying the ticket key is created in Git and pushed to GitHub, and a pull request starts the GitHub Actions continuous integration pipeline, which runs the tests and builds the Docker images. After review and a green pipeline, the branch is merged into main, which starts the deployment workflow that builds the tagged images and rolls them out to the Kubernetes cluster. Prometheus and Grafana then show the health, traffic and latency of the new version, the Ansible-provisioned server hosts the cluster, and finally the Jira ticket is moved to Done.

```
Jira ticket -> Git branch -> Pull request -> CI -> Merge -> CD -> Kubernetes ->
Prometheus/Grafana -> Jira Done
```

Together, the activities show the complete path from a planned requirement to a monitored deployment.

3.2 Discussion

- Reproducibility: containers and Docker Compose give every team member the same environment, and one command starts all eight services.
- Visibility of the system: the health endpoint, Prometheus targets and the Grafana dashboard show immediately whether the services are up, how many requests they handle and how fast they respond.
- Traceability: Jira keys in commit messages tie each code change to a planned work item, and the Jira reports show progress by status, type and epic.
- Quality gate and resilience: CI runs the tests and builds on every change, Kubernetes probes restart unhealthy pods and the autoscaler adds backend replicas under load.
- Repeatable infrastructure: the Ansible roles rebuild a server in the same way each time.

Limitations and future work

- Secrets such as the JWT key are stored in configuration files and should move to a secret manager.
- Alerting rules (Alertmanager) and a separate lint stage can be added to the monitoring and CI setup.
- The Airflow retraining DAG simulates some steps and should call the real training script.

Chapter 4: Conclusion

4.1 Conclusion

The Smart Cricket Tournament Tracker shows that a single application can pass through the whole Agile and DevOps toolchain. The work was planned in Jira sprints, stored in Git and GitHub, packaged into multi-stage Docker images, run as a multi-service stack with Docker Compose, deployed to Kubernetes, validated and delivered through GitHub Actions, provisioned with Ansible and monitored with Prometheus and Grafana. The screenshots confirm that the platform runs end to end: live scoring, standings, player analytics and machine-learning predictions work together, and their health and performance can be observed in real time. The project therefore meets the objectives of the Agile Software Development and DevOps course.

Chapter 5: References

[1] Docker Documentation, https://docs.docker.com
[2] Docker Compose Documentation, https://docs.docker.com/compose
[3] Kubernetes Documentation, https://kubernetes.io/docs
[4] K3s Documentation, https://docs.k3s.io
[5] GitHub Actions Documentation, https://docs.github.com/actions
[6] Ansible Documentation, https://docs.ansible.com
[7] Prometheus Documentation, https://prometheus.io/docs
[8] Grafana Documentation, https://grafana.com/docs
[9] Atlassian Jira Software Documentation, https://support.atlassian.com/jira-software-cloud
[10] Pro Git, Scott Chacon and Ben Straub, https://git-scm.com/book
[11] FastAPI Documentation, https://fastapi.tiangolo.com
[12] MLflow Documentation, https://mlflow.org/docs/latest
[13] Apache Airflow Documentation, https://airflow.apache.org/docs